

university of groningen

Performance Engineering: Lecture #01

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-08-31

Overview



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Content



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Course Practices



- **Questions are welcome at any time!**
  - Do not only consume information, reason about concepts
- **On-site presence is recommended**
  - No video broadcasting or recording is foreseen
- **On-site presence during project presentations is mandatory**
  - Exceptions need to be agreed ahead of the sessions
- **Assignments will be performed in groups**
  - *Remember*: During the exam you are at your own

Course Objective



- The purpose of this course is to learn **fundamental aspects and techniques** used for performance engineering
  - We will look at specific cases to create a better understanding
  - Technical details, e.g. about tools, will not be assessed during the exam

Course Topics



The course will cover the following topics

1. Performance and performance measurement
2. Compilers
3. Performance models
4. Optimizations at the processor core level
5. Optimizations at the memory subsystem level
6. Data structures
7. Optimization of parallel applications
8. Optimization for accelerators
9. I/O optimization

Available Experience



- **Question**: How many course participants have experience with the following technologies?
  - OpenMP
  - MPI
- **Note**: C or C++ programming experience is assumed

Intended Learning Outcomes (ILOs)



- **ILO #1**: Apply and assess different established optimization techniques
- **ILO #2**: Demonstrate and compare several performance modelling methods, and assess their usefulness for practical problems with focus on assessing attainable performance for a given application and computer architecture
- **ILO #3**: Design an empirical performance analysis process for any application, interpret its results, and recommend solutions for performance improvement
- **ILO #4**: Use different performance engineering techniques and tools including performance analysis tools and benchmarking techniques

Course Assessment



- For assessing your performance, you will have to accomplish the following tasks:
  - **Task #1**: Follow lab assignments
  - **Task #2**: Project
  - **Task #3**: The final exam
- Tasks #1 and #2 will be done in groups of 2 students

How are the ILOs Assessed?

	Task #1	Task #2	Task #3
ILO #1	✔	✔	✔
ILO #2	✔		✔
ILO #3		✔	
ILO #4	✔	✔	

Task #1: Lab Assignments



- One lab assignment per week
- They are graded P/F
- One resubmission is possible

Task #2: Project



- Performance analysis, modelling, and optimization for an application of choice
- Results should be documented in a report and presented during the last week
- Both report and presentation will be graded 1-10
  - Weight for final grade: 25% and 15%

Use of AI



- Use of Large Language Model (LLM) services (such as ChatGPT, DeepSeek etc.):
  - Is **allowed** for text polishing
  - Is **not allowed** for generation of full-text blocks
  - Is **allowed** for code generation, but you need to
    - Verify correctness
    - Proof, analyse, and explain performance behaviour
- You remain fully responsible for the submitted content
- Specific rules for using LLMs:
  - Add a reference with name and version of the LLM
  - Add a clarification if a paid version is used
  - Document all prompts
- Otherwise: **Reporting on experiences with AI is encouraged**

Task #3: Exam



- The final exam covers all the topics presented during the course
- It is graded 1-10
- Exam structure:
  - Multiple-choice and open questions
  - All 9 course topics will be covered
- Open questions will receive more points

Course Practices: Labs and Project



- Practical work to be implemented on device of your choice
  - Typically your own device
    - Installing software will be your responsibility
  - Access to an HPC cluster will be provided
    - Register for account in this Google Sheet
      IMPORTANT: Enter key using the OpenSSH key format
- **Documentation is key** to guarantee **reproducibility** of the results
  - Code
  - Hardware
  - Development environment (e.g., compiler)



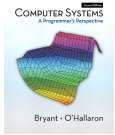
Course Text Books

- D. Bakhvalov, “Performance Analysis and Tuning on Modern CPUs,” 2nd edition, 2024. [Online]. Available: `https://github.com/dendibakh/perf-book/releases`

- R. E. Bryant and D. R. O’Hallaron, “Computer Systems: A Programmer’s Perspective,” 2nd edition, Pearson, 2011

- Georg Hager and Gerhard Wellein, “Introduction to high performance computing for scientists and engineers,” 1st edition, CRC, 2010. doi: 10.1201/EBK1439811924.







Content



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Definition



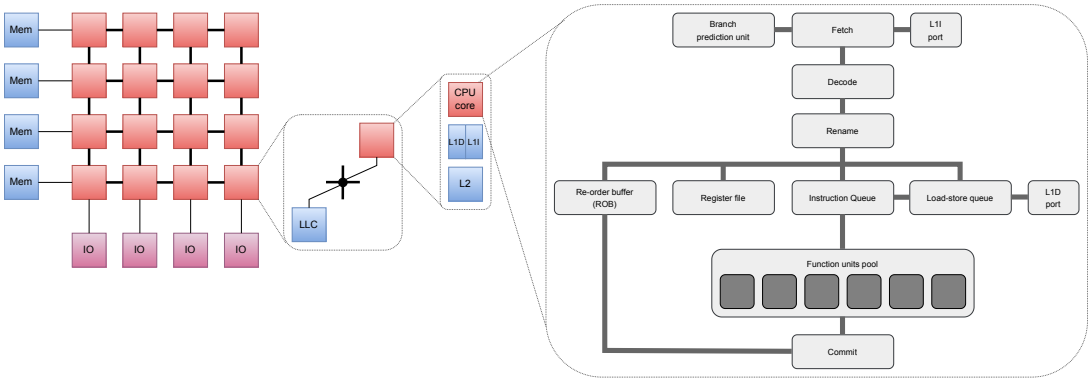
- **Computing system** = Integrated set of hardware and software components designed to transport, process, and store data
- Selected typical components
  - Central Processing Unit (CPU)
  - Storage hardware
  - Network hardware
  - Operating system
  - Compilers
  - ...

Complex Systems



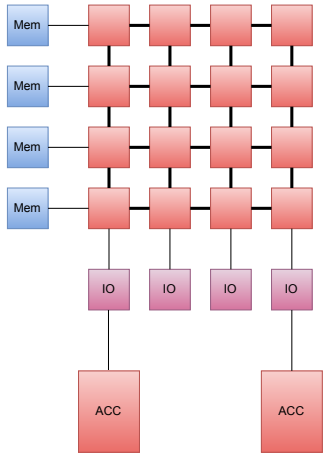
- **Complex system** = A system composed of multiple components or constituents that interact
- The interaction leads to properties of this system that cannot be easily inferred from the single components or constituents
- **Computing systems are complex systems**

Example: Many-Core CPU Architecture

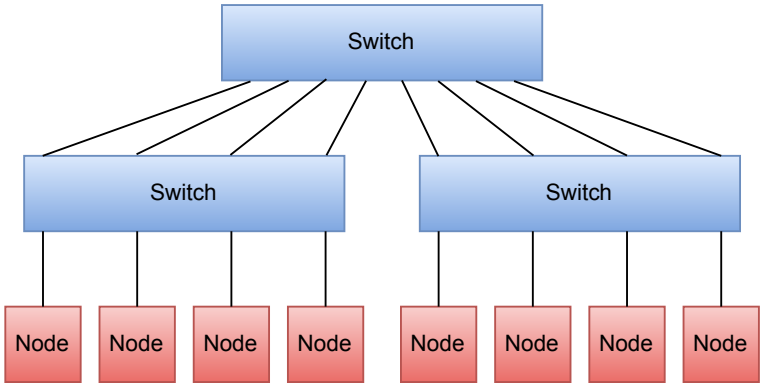


Accelerator

- Accelerators are mostly discrete and attached via an I/O interface to a CPU
- Examples:
  - GPU
  - TPU
  - Neuromorphic devices

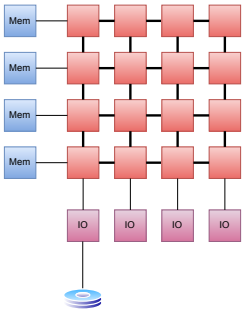


Multi-Node Systems

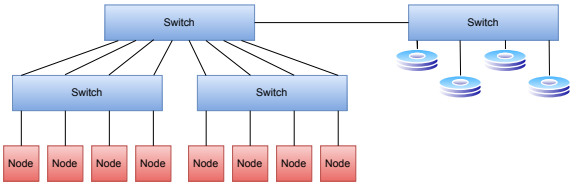


Storage

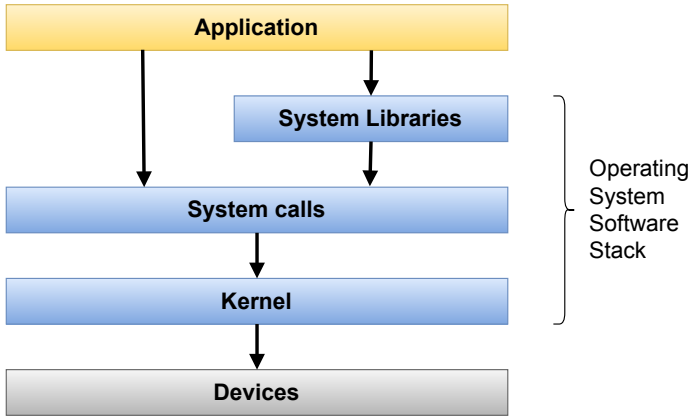
Node-local storage:



System-level storage:



System Software



Content



1. Course Information
2. Computing Systems Organization
3. **Performance Engineering**
4. Performance and Performance Metrics
5. Final

Definition



- **Performance Engineering** = Systematic endeavour to enhance the efficiency of software applications that satisfy all functional requirements
- Differing definitions of efficiency are permissible under this definition
  - **Performance** = Efficient use of the target system
  - **Energy efficiency** = Work per energy unit
- Performance engineering efforts should **not change the functional behaviour** of the software application

Importance of Performance Engineering



- Example speed-ups from performance engineering:
  A program that multiplies two 4096×4096 matrices

[Leiserson et al.; 2020]

Version	Implementation	Running time (s)	GFLOPS	Absolute speedup	Relative speedup	Fraction of peak (%)
1	Python	25,552.48	0.005	1	–	0.00
2	Java	2,372.68	0.058	11	10.8	0.01
3	C	542.67	0.253	47	4.4	0.03
4	Parallel loops	69.80	1.969	366	7.8	0.24
5	Parallel divide and conquer	3.80	36.180	6,727	18.4	4.33
6	plus vectorization	1.10	124.914	23,224	3.5	14.96
7	plus AVX intrinsics	0.41	337.812	62,806	2.7	40.45

Methodologies Overview

- Many different approaches to performance engineering have been identified in the literature
- Different approaches can be combined

☞ **There is no unique approach to performance engineering**

[Brendan Gregg; Prentice Hall; 2013]

Methodology	Type
Streetlight anti-method	observational analysis
Random change anti-method	experimental analysis
Blame-someone-else anti-method	hypothetical analysis
Ad hoc checklist method	observational and experimental analysis
Problem statement	information gathering
Scientific method	observational analysis
Diagnosis cycle	analysis life cycle
Tools method	observational analysis
USE method	observational analysis
Workload characterization	observational analysis, capacity planning
Drill-down analysis	observational analysis
Latency analysis	observational analysis
Method R	observational analysis
Event tracing	observational analysis
Baseline statistics	observational analysis
Performance monitoring	observational analysis, capacity planning
Queueing theory	statistical analysis, capacity planning
Static performance tuning	observational analysis, capacity planning
Cache tuning	observational analysis, tuning
Micro-benchmarking	experimental analysis
Capacity planning	capacity planning, tuning

Ad-Hoc Checklist Methodology



[Brendan Gregg; Prentice Hall; 2013]

- **Approach**: Process a **checklist** with each entry of the type:
  Run application X and if observation O, then take action A_1 or A_2 or ...
- Possible information contained in checklist entries
  - Type of performance tool to run
  - Instructions on how to interpret the results
  - Suggested actions
- **Advantages**: Easy to follow, can be fast, can be prescriptive
- **Disadvantages**: Limited to known performance issues, risk of missing **hidden parameters**

Problem Statement Methodology



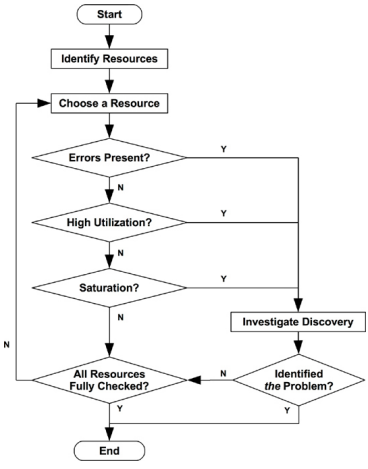
[Brendan Gregg; Prentice Hall; 2013]

- **Approach**: Check for **possible problems** through a **structured list of questions**:
  - What makes you think there is a performance problem?
  - Has this system ever performed well?
  - What has changed recently?
  - Can the performance degradation be expressed in terms of latency or run time?
- **Advantages**: Fast, can resolve some (more often occurring) issues
- **Disadvantages**: Limited scope

USE Methodology

- Investigate
  - **Utilization**: To what extent are relevant resources utilized?
  - **Saturation**: Do requests have to wait before being able to use a resource?
  - **Errors**: How many errors do occur?
- Resources may be both hardware and software resources

[Brendan Gregg; Prentice Hall; 2013]



Scientific Methodology



[Brendan Gregg; Prentice Hall; 2013]

1. **Question**: Performance problem statement
   - Example: Why is throughput of instructions low?
2. **Hypothesis**: Possible root cause for this problem
   - Example: There are chains of instructions that depend on each other (long critical paths)
3. **Prediction**: Measures that remedy this root cause
   - Example: Reorganization of the code resulting in shorter critical paths
4. **Test**: Perform runs after measures have been applied
5. **Analysis**: Assess the impact of the measures
   - Possibly need to go back to step #2

Model-based Methodology



1. **Measurement**: Measure performance and apply performance analysis techniques to identify performance critical regions
2. **Modelling**: Develop performance models to determine attainable performance
3. **Verification**: Verify whether model is sufficiently accurate
   - May have to go back to step #2
4. **Analysis**: Analyse deviations between observed and predicted performance
5. **Optimize**: Develop and implement strategy to address performance issues

Optimization versus Improvement



- **Optimization** = Process of modifying an application and/or the considered system with the goal of enlarging application performance **towards a defined optimum**

- **Improvement** = Process of modifying an application and/or the considered system with the goal of enlarging application performance **with respect to a reference performance**

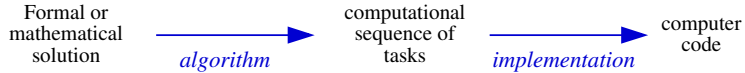
Content



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Problem-Solving Abstraction

- Numerical problem-solving abstraction



- Execution of computer code = computation
- What is computation?
  - Process of manipulating data according to simple rules
  - Let data be represented by $w \in \{0,1\}^n$

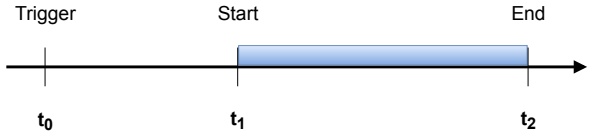
$$w^{(0)} \to w^{(1)} \to \cdots \to w^{(i)} \to \cdots$$

Events



- An event refers to a point in time where something happens
  - An event has no duration
- Examples:
  - Instruction is scheduled
  - Change of a signal value (e.g. indicating arrival of data)
  - Arrival of a message
  - Change of a state
  - A counter reaches/exceeds a given value

Latency



- **Latency** = Time from trigger of an operation until its completion
  - Here: $\Delta t = t_2 - t_0$
  - Example: Time from issue of a load instruction until arrival of the data at the destination
  - Also known as **turnaround time** or **time-to-solution**
- **Start-up latency** = Time from trigger of an operation until effects become visible
  - Here: $\Delta \tau = t_1 - t_0$
  - Example: Time from issue of a message receive command until arrival of first data
  - Also known as **response time**

Wall-Clock Time



- **Wall-clock time** = Time from a start event until an end event
  - Also known as **elapsed real time**
  - Examples: Start and completion of an application or task
  - Special case of latency

Resource Utilization Time



- **Resource utilization time** = Accumulated time between two events during which a resource is in use
- The reference clock is not incremented while the resource is not in use
- **Example**: Time during which a process or thread uses a CPU core

Performance, Bandwidth, and Throughput



- **Performance** = Amount of work W per time unit

$$b(W) = \frac{W}{\Delta t(W)}$$

  - While this definition does not specify work, it needs to be well-defined, e.g. execution of a specific **workload** (also called problem size)

- **Bandwidth** = Amount of data passing a **particular interface** per time unit
  - It is important to define this interface

- **Throughput** = Amount of work items processed per time unit
  - Example: Number of floating-point operations per time unit

$$B_{\mathrm{fp}} = \frac{N_{\mathrm{fp}}}{\Delta t}$$

Algorithmic Versus Machine Performance



- Performance of an application may be increased by
  - Improving the **algorithmic performance**
    - This is not considered in this lecture
  - Improving the **machine performance**
    - Focus of this lecture for given hardware architecture and system software stack

Content



1. Course Information
2. Computing Systems Organization
3. Performance Engineering
4. Performance and Performance Metrics
5. Final

Summary Lecture #01



- Computing systems
  - Complex systems comprising different hardware and software components
- Performance and performance metrics
  - Definition of various terms: Latency, start-up latency, performance, bandwidth, throughput
  - Wall-clock versus resource utilization time
- Performance engineering
  - Definition of performance engineering and different approaches to performance engineering

Questions?



[Wiki Media]